

MasterClass in DSA

Exercise Set - 5

Topic: Doubly Circular Linked List

(Exercises where solutions differ from Doubly Linked List)

Total Exercises: 20

Easy: 5 | Intermediate: 9 | Difficult: 6

*CoreCode Programming Academy
An Academy by Yogeshwar Shukla*

General Instructions

1. This exercise set focuses on operations where DCLL provides significant advantages over DLL.
2. Use your existing DCLL implementation (list.h, list.c) as the base.
3. Key DCLL property: `p_list->prev` points to last node, `p_list->next` points to first node.
4. Empty list condition: `p_list->prev == p_list && p_list->next == p_list`
5. Traversal terminates when: `p_run == p_list` (returned to head sentinel).
6. Leverage $O(1)$ access to both ends for efficient implementations.

Key Complexity Differences: DLL vs DCLL

insert_end: DLL = $O(n)$, DCLL = $O(1)$ via `p_list->prev`

get_end: DLL = $O(n)$, DCLL = $O(1)$ via `p_list->prev->data`

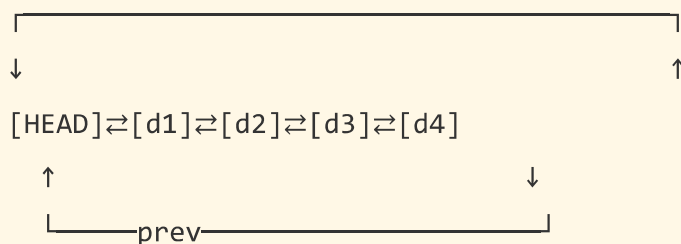
pop_end: DLL = $O(n)$, DCLL = $O(1)$ via `p_list->prev`

remove_end: DLL = $O(n)$, DCLL = $O(1)$ via `p_list->prev`

Backward traversal: DLL = must find last first $O(n)$, DCLL = start from `p_list->prev`

Concatenation: DLL = $O(n)$ to find last, DCLL = $O(1)$ pointer manipulation

DCLL Structure Visualization



HEAD->next = first data node (`d1`), **HEAD->prev** = last data node (`d4`)

Section A: Easy Exercises

Exercise 1: Get Second Last Element - O(1) EASY

Write a function to retrieve the second last element of the list in O(1) time. This is possible in DCLL because `p_list->prev->prev` gives direct access. In DLL, this would require O(n) traversal.

```
status_t get_second_last(list_t* p_list, data_t* p_data)
```

Sample Input:

[HEAD] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ [40] ⇄ ⌚

Sample Output:

*p_data = 30, returns SUCCESS

Complexity: DCLL = O(1), DLL = O(n)

Exercise 2: Swap First and Last Elements - O(1) EASY

Write a function to swap the data of the first and last nodes in O(1) time. Use `p_list->next` for first and `p_list->prev` for last.

```
status_t swap_first_last(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ [40] ⇄ ⌚

Sample Output:

[HEAD] ⇄ [40] ⇄ [20] ⇄ [30] ⇄ [10] ⇄ ⌚

Complexity: DCLL = O(1), DLL = O(n) to find last

Exercise 3: Move Last to First - O(1) EASY

Write a function to move the last node to become the first node in O(1) time. This is essentially a right rotation by 1. Use pointer manipulation, not data copying.

```
status_t move_last_to_first(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ [40] ⇄ ⌚

Sample Output:

[HEAD] → [40] → [10] → [20] → [30] → ∅

Complexity: DCLL = $O(1)$, DLL = $O(n)$

Exercise 4: Move First to Last - $O(1)$ EASY

Write a function to move the first node to become the last node in $O(1)$ time. This is essentially a left rotation by 1. Use pointer manipulation, not data copying.

```
status_t move_first_to_last(list_t* p_list)
```

Sample Input:

[HEAD] → [10] → [20] → [30] → [40] → ∅

Sample Output:

[HEAD] → [20] → [30] → [40] → [10] → ∅

Complexity: DCLL = $O(1)$, DLL = $O(n)$

Exercise 5: Check if Sum of First and Last Equals Target EASY

Write a function that returns TRUE if the sum of first and last elements equals a given target, FALSE otherwise. Leverage $O(1)$ access to both ends.

```
status_t first_last_sum_equals(list_t* p_list, data_t target)
```

Sample Input:

[HEAD] → [10] → [20] → [30] → [40] → ∅, target = 50

Sample Output:

TRUE (10 + 40 = 50)

Section B: Intermediate Exercises

Exercise 6: Efficient Rotate Right by K INTERMEDIATE

Write a function to rotate the list right by K positions in $O(K)$ time. In DCLL, this only requires adjusting where the "logical start" is. Compare this to DLL where finding the split point requires $O(n)$ traversal.

```
status_t rotate_right_efficient(list_t* p_list, int k)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌⊙, k = 2

Sample Output:

[HEAD]⇌[4]⇌[5]⇌[1]⇌[2]⇌[3]⇌⊙

Hint: Move backward from `p_list->prev` K times to find new last, then adjust head's next and prev.

Exercise 7: Efficient Rotate Left by K INTERMEDIATE

Write a function to rotate the list left by K positions in $O(K)$ time. Move forward from `p_list->next` K times to find new first.

```
status_t rotate_left_efficient(list_t* p_list, int k)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌⊙, k = 2

Sample Output:

[HEAD]⇌[3]⇌[4]⇌[5]⇌[1]⇌[2]⇌⊙

Exercise 8: Insert Before Last - $O(1)$ INTERMEDIATE

Write a function to insert a new node just before the last node in $O(1)$ time. This becomes the new second-to-last element.

```
status_t insert_before_last(list_t* p_list, data_t new_data)
```

Sample Input:

[HEAD]⇌[10]⇌[20]⇌[30]⇌⊙, new_data = 25

Sample Output:

[HEAD]⇌[10]⇌[20]⇌[25]⇌[30]⇌⊙

Approach: Use `generic_insert(p_list->prev->prev, new_node, p_list->prev)`

Exercise 9: Delete Second Last - O(1) INTERMEDIATE

Write a function to delete the second last node in O(1) time.

```
status_t remove_second_last(list_t* p_list)
```

Sample Input:

[HEAD] → [10] → [20] → [30] → [40] → ∅

Sample Output:

[HEAD] → [10] → [20] → [40] → ∅

Approach: Use `generic_delete(p_list->prev->prev)`

Exercise 10: Traverse N Rounds Bidirectionally INTERMEDIATE

Write a function that alternately traverses the list: first forward one complete round, then backward one complete round, repeating N times. Print elements during each traversal. This showcases the circular and bidirectional nature together.

```
void traverse_n_rounds_alternate(list_t* p_list, int n)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌⊙, n = 2

Sample Output:

Forward: 1 2 3, Backward: 3 2 1, Forward: 1 2 3, Backward: 3 2 1

Exercise 11: Split into Front and Back Halves - O(n) INTERMEDIATE

Write a function to split the list into two halves. Both resulting lists must be valid DCLLs (circular). The original list becomes the front half. In DCLL, connecting the circular links requires careful pointer work.

```
list_t* split_into_halves(list_t* p_list)
```

Sample Input:

[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌[6]⇌⊙

Sample Output:

Front: [HEAD]⇌[1]⇌[2]⇌[3]⇌⊙, Back: [HEAD]⇌[4]⇌[5]⇌[6]⇌⊙

Note: Both lists must have their circular links properly established.

Exercise 12: Interleave Two DCLLs INTERMEDIATE

Write a function to interleave two DCLLs: take alternate nodes from each list. The first list is modified in-place, the second becomes empty. Result must remain a valid DCLL.

```
status_t interleave_lists(list_t* p_list_1, list_t** pp_list_2)
```

Sample Input:

List1: [1]⇌[3]⇌[5]⇌⊙, List2: [2]⇌[4]⇌[6]⇌⊙

Sample Output:

List1: [1]⇌[2]⇌[3]⇌[4]⇌[5]⇌[6]⇌⊙, List2: destroyed

Exercise 13: Check Symmetry Using Both Ends INTERMEDIATE

Write a function to check if the list is symmetric (palindrome) by comparing from both ends simultaneously. Use `p_list->next` moving forward and `p_list->prev` moving backward until they meet or cross.

```
status_t is_symmetric(list_t* p_list)
```

Sample Input:

[HEAD]⇨[1]⇨[2]⇨[3]⇨[2]⇨[1]⇨⊙

Sample Output:

TRUE

Advantage: In DCLL, backward pointer starts immediately at last node. In DLL, you first need $O(n)$ to find last.

Exercise 14: Find Pairs with Sum from Both Ends INTERMEDIATE

Given a sorted DCLL, find all pairs of elements that sum to a target value. Use two pointers: one from start (`p_list->next`) and one from end (`p_list->prev`), moving towards each other.

```
void find_pairs_with_sum(list_t* p_list, data_t target)
```

Sample Input:

[HEAD]⇨[1]⇨[2]⇨[4]⇨[5]⇨[7]⇨[8]⇨⊙, target = 9

Sample Output:

Pairs: (1, 8), (2, 7), (4, 5)

Complexity: $O(n)$ with two-pointer technique leveraging $O(1)$ access to both ends.

Section C: Difficult Exercises

Exercise 15: Josephus Problem - Bidirectional DIFFICULT

Implement a variant of the Josephus Problem where elimination alternates direction: first K steps forward, then K steps backward, then forward again, etc. DCLL makes backward traversal trivial.

```
int josephus_bidirectional(int n, int k)
```

Sample Input:

`n = 5 (people), k = 2`

Sample Output:

`Survivor position (1-indexed)`

Process: Start at 1, count 2 forward (eliminate 2), count 2 backward from there (eliminate ?), and so on.

Exercise 16: Zigzag Rearrangement Using Both Ends DIFFICULT

Rearrange the list in zigzag pattern: first, last, second, second-last, third, third-last, etc. Use simultaneous traversal from both ends, leveraging DCLL's $O(1)$ access to last node.

```
status_t zigzag_rearrange(list_t* p_list)
```

Sample Input:

`[HEAD]⇌[1]⇌[2]⇌[3]⇌[4]⇌[5]⇌[6]⇌⊙`

Sample Output:

`[HEAD]⇌[1]⇌[6]⇌[2]⇌[5]⇌[3]⇌[4]⇌⊙`

Approach: Extract from front and back alternately, rebuilding the list.

Exercise 17: Circular Deque Implementation DIFFICULT

Implement a double-ended queue (deque) using your DCLL where all four operations are $O(1)$: `insert_front`, `insert_rear`, `delete_front`, `delete_rear`. Also implement `get_front` and `get_rear`.

```
typedef list_t deque_t;
deque_t* create_deque(void);
status_t deque_insert_front(deque_t* p_dq, data_t data);
status_t deque_insert_rear(deque_t* p_dq, data_t data);
status_t deque_delete_front(deque_t* p_dq, data_t* p_data);
status_t deque_delete_rear(deque_t* p_dq, data_t* p_data);
status_t deque_get_front(deque_t* p_dq, data_t* p_data);
```

```
status_t deque_get_rear(deque_t* p_dq, data_t* p_data);  
status_t is_deque_empty(deque_t* p_dq);
```

All operations:

O(1) time complexity

Note: This is a wrapper over DCLL operations. In DLL, delete_rear and get_rear would be O(n).

Exercise 18: Round Robin with Priority Boost **DIFFICULT**

Implement an enhanced round-robin scheduler: processes cycle in a DCLL. Each process has a remaining burst time. Each quantum, the current process runs. If a process waits a full round without running, its priority increases (it jumps to front). Use DCLL's circular nature for continuous cycling and bidirectional movement.

```
void priority_round_robin(list_t* p_process_list, int quantum, int boost_threshold)
```

Sample:

Processes with varying burst times cycle; starved processes get boosted to front

DCLL Advantage: Moving a node to front is $O(1)$, cycling is natural, backward check for starvation is easy.

Exercise 19: Reverse in Pairs from Both Ends **DIFFICULT**

Write a function that simultaneously reverses pairs from both ends, meeting in the middle. First, swap pairs at the front (1st-2nd), then swap pairs at the back (last-second_last), then move inward. This is elegant in DCLL due to $O(1)$ access to both ends.

```
status_t reverse_pairs_both_ends(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [1] ⇄ [2] ⇄ [3] ⇄ [4] ⇄ [5] ⇄ [6] ⇄ [7] ⇄ [8] ⇄ ∅

Sample Output:

[HEAD] ⇄ [2] ⇄ [1] ⇄ [4] ⇄ [3] ⇄ [6] ⇄ [5] ⇄ [8] ⇄ [7] ⇄ ∅

Exercise 20: Maximum Sum Circular Sublist **DIFFICULT**

Find the maximum sum of a contiguous sublist in a circular list. The sublist can wrap around from end to beginning. Use the circular nature: maximum circular sum = $\max(\text{normal max subarray sum}, \text{total_sum} - \text{min subarray sum})$.

```
data_t max_circular_sublist_sum(list_t* p_list)
```

Sample Input:

[HEAD] ⇄ [8] ⇄ [-4] ⇄ [3] ⇄ [-2] ⇄ [5] ⇄ ∅

Sample Output:

14 (sublist: 5, 8, -4, 3, -2 + wrap doesn't help; or 3, -2, 5, 8 = 14)

Algorithm: Use modified Kadane's algorithm. The circular wrap case is handled by finding minimum subarray and subtracting from total.

Summary: DLL vs DCLL Key Differences

Operations with Different Complexities

Operation	DLL	DCLL
insert_end	$O(n)$	$O(1)$
get_end	$O(n)$	$O(1)$
pop_end	$O(n)$	$O(1)$
remove_end	$O(n)$	$O(1)$
get_second_last	$O(n)$	$O(1)$
concat_lists	$O(n)$	$O(1)$
Start backward traversal	$O(n)$	$O(1)$

Code Pattern Differences

DLL - Finding last node:

```
p_run = p_list;  
while(p_run->next != NULL) p_run = p_run->next;
```

DCLL - Accessing last node:

```
p_last = p_list->prev; //  $O(1)$  direct access
```

Traversal Termination

DLL: while(p_run != NULL)

DCLL: while(p_run != p_list) or use do-while for at least one iteration

Applications Best Suited for DCLL

Deque (Double-Ended Queue): All four operations $O(1)$

Round Robin Scheduling: Natural cycling without end checks

Josephus Problem: Circular elimination with bidirectional movement

Music/Video Playlist: Loop with prev/next track $O(1)$

Browser History: Navigate back and forward $O(1)$

Undo/Redo System: Bidirectional operation stack

Two-Pointer on Sorted Data: Start from both ends efficiently